

PROJECT REPORT · C LANGUAGE CONSOLE APPLICATION



Smart Hospital & Resource Allocation System

Patient triage · Ward & bed management · Billing · Analytics

CSC 1012 Introduction to Computer Programming

University of Sri Jayewardenepura



Student Details

NAME

K.T.D.K.Himesh Wijethunga

STUDENT ID

AS20250647

SUBMITTED DATE

2026/Sep/20

GITHUB REPOSITORY

<https://github.com/kwh440/Smart-Hospital-Resource-Allocation-System>

Content

1. Introduction	3
2. Objectives	3
3. Project Overview	3
3.1 Assumptions Made During Development	3
4. Methodology	4
5. How I Planned My Project	5
6. Methods Followed for the Planning	5
7. How GitHub Was Used and Best Practices Followed	6
7.1 GitHub Repository	6
7.2 Best Practices Followed	7
8. Implementation	7
8.1 High Level Architecture	7
8.2 Program Lifecycle & Menu Map	7
8.3 Core Business Rules	8
8.4 Implemented Functions (58)	8
8.5 Data Structure Designs	10
8.6 Persistence File Formats	11
9. Conclusion	12
10. Appendix	12
10.1 GitHub Repository	12
10.2 Source Code	12
10.3 Output Screenshots	13
10.4 Project Details	16
11. Challenges Faced	16
12. System Requirements	16

1 Introduction

A hospital has to decide, quickly and repeatedly, who needs care first, where each patient should stay and what the stay will cost. The **Smart Hospital & Resource Allocation System** is a menu driven console application written in C that supports those decisions: it registers patients, ranks them by urgency, recommends and allocates ward beds, produces itemised bills and summarises hospital performance.

The project applies structured, modular C programming (structs, arrays, functions and file I/O) to a realistic problem and is version controlled on GitHub. It runs in a terminal and keeps its data in plain text files, so no database or external library is needed.

8

source modules

58

functions

4 / 45

wards / beds

2.8k

lines of C

2 Objectives

The objectives of this project are:

1

Patient registration and triage capture patient details, block duplicates and rank patients by three urgency levels.

2

Ward and bed allocation recommend a ward from age and urgency, assign the first free bed and release it on discharge.

3

Automated billing compute base fees, emergency surcharges, ward costs and age subsidies, and export itemised receipts.

4

Performance analytics report the triage mix, revenue, bed occupancy and the highest paying patient.

5

Data persistence save and reload patient and bed records between runs using plain text files.

6

Modular C programming split responsibilities into header / source pairs with validated input and clear function contracts.

7

Version control and documentation use GitHub to track the work and document the design, outputs and results in this report.

3 Project Overview

The application presents a 13 option menu grouped into three areas. It loads saved data at startup, loops until the user chooses option 13, and then writes all records back to disk. All state is held in fixed size in memory arrays.

PATIENT MANAGEMENT & TRIAGE	WARD & BED MANAGEMENT	FINANCIALS & ANALYTICS
• Register new patients with duplicate check • Directory in arrival order • Priority triage queue (Level 3 → 1) • Search by ID or partial name • Update patient records	• Doctor specialties directory • Ward list with live availability • Bed matrix and ward status tables • Bed allocation with ward recommendation • Discharge and bed release	• Performance and analytics report • Itemised bill with surcharge and subsidy • Receipt export to text file • Save data and exit

3.1 Assumptions Made During Development

Environment & scope

- A **single user** operates the program at a time from one console; there is no login, role or concurrency handling.
- The program runs from a folder where a data/ subfolder can be created, in a terminal with ANSI colour and Unicode box drawing support. The Windows console is the primary target (ANSI mode and UTF-8 are enabled explicitly); the same code also builds on Linux through conditional compilation.

Capacity & identifiers

- Capacity is fixed at **100 patients, 45 beds, 4 wards and 7 specialties**; registration is refused once 100 patients exist.
- Patient IDs are sequential (PAT-1001, PAT-1002 ...). Patients are never deleted, so IDs stay unique.
- Two records with the **same name and contact number** are treated as the same person and the second registration is blocked.
- Each patient occupies at most **one bed** at a time; a patient with a bed cannot be allocated a second one.

Clinical & billing rules

- Urgency is entered by staff on a three level scale (Normal, Urgent, Critical); Level 3 patients receive immediate attention (0 minute wait).
- Ward recommendation is advisory only; the user can always choose another ward. Beds are assigned first fit (lowest free ID).
- All money values are in **LKR**. One base consultation fee is charged per bill; stay length is entered in whole days (0-365) and can be corrected at billing time.
- The 15 % age subsidy applies to the gross total for patients younger than 5 or older than 65. Outpatients (no ward) pay no ward cost.

Persistence & known limits

- Data is saved only when the user chooses **option 13**; closing the window without it discards changes made in that session.
- Text fields must not contain a semicolon (the file delimiter), and a saved patient line is assumed to fit in 255 characters.
- Discharging a patient frees the bed and resets their ward and bed to 0, so a bill or report generated *after* discharge treats the stay as outpatient. Bills are therefore assumed to be calculated **before** discharge.
- Contact numbers are stored as free text and are not format checked.

4

Methodology

The system was developed in C and organised around the following design decisions:

- **Modular decomposition** eight source units, each exposing its public interface through a header file; `main.c` only orchestrates.
- **Data first design** domain records are C structs stored in fixed size global arrays; relationships use ID values rather than pointers.
- **Rule based logic** triage ordering, ward recommendation and billing use explicit formulas that are easy to test.
- **Defensive input handling** bounded integer and non empty string readers reprompt until the input is valid.
- **Plain text persistence** semicolon delimited files are loaded at startup and saved on exit.
- **Separated presentation** colours, animations and charts live in `ui_effects.c`, apart from business logic.

Development workflow and tools

Environment	Code::Blocks IDE with the GCC (MinGW) compiler on Windows
Way of working	Built in small steps: write one module, compile it, test it from the menu, then move on to the next
Compiler checks	Compiled with warnings enabled (<code>-Wall</code>) and fixed each warning before continuing
Version control	Git and GitHub: a feature branch for each module or task, merged into <code>main</code> once working
Documentation	Every function has a header comment giving its purpose, inputs and return value
Code comments	Comments were added with the help of Gemini 3.6 Flash in Antigravity, on a dedicated <code>comment_branch</code>

Testing approach

The program was tested manually through the menu with sample patients, and the results were checked against hand calculations.

Test	Expected result
Invalid input (letters, out of range numbers, empty text)	The prompt is repeated until valid input is entered
Register the same name and contact number twice	The second registration is blocked as a duplicate
Allocate a bed to a Critical patient	ICU is recommended, a free bed is assigned and the bed matrix updates
Bill a Critical Cardiology patient with 4 ICU days	LKR 4,500 + 2,250 surcharge + 100,000 ward cost = LKR 106,750
Save with option 13, then restart the program	Patients, beds and ward counters are restored

5 How I Planned My Project

I planned the project in five stages before writing most of the code:

Stage	What I did
1. Understanding the problem	Read the assignment requirements and listed what a small hospital system must do: register patients, prioritise emergencies, manage beds, calculate bills and report.
2. Identifying entities and features	Found four main entities (patient, ward, bed and doctor specialty) and grouped the features into 13 menu options in three areas.
3. Designing the data	Chose structs and fixed size arrays to hold the entities, and decided how they refer to each other using ID numbers.
4. Splitting into modules	Divided the program into eight source files, each with one clear responsibility and its own header file.
5. Choosing tools and milestones	Selected Code::Blocks, GCC and GitHub, and scheduled the work in small milestones: data structures, patients, wards and beds, billing, reports, file handling, then polish and documentation.

6 Methods Followed for the Planning

I followed a simple top down, step by step approach:

Method	How it was applied
Top down design	Started from the main menu and broke each option into smaller functions. For example, option 12 is built from small formula functions such as <code>calculateEmergencySurcharge</code> and <code>calculateAgeSubsidy</code> .
Pseudocode and flow planning	Wrote the key rules (triage order, ward recommendation and billing) as pseudocode and simple flowcharts before coding them.
Incremental development	Implemented one module at a time, compiling and testing after each step so that errors were found early.
Modular programming	Used separate header and source files with clear function prototypes, and shared the main data arrays between modules.
Reusable helpers	Wrote the input validation functions once and reused them wherever the program reads user input.
Review and refinement	Fixed bugs after testing and improved the console interface (colours, tables and animations) last, so it could not affect the logic.

7 How GitHub Was Used and Best Practices Followed

7.1 GitHub Repository

Repository URL	https://github.com/kwh440/Smart-Hospital-Resource-Allocation-System
Repository name	Smart-Hospital-Resource-Allocation-System
Branch	main (default), plus feature branches for modules and tasks (see 7.1)
Project	Smart Hospital & Resource Allocation System (C)

Branch structure

Development followed a simple feature branch workflow. The stable, working version lives on the **main** branch, while each module or task was developed on its own branch and merged into **main** once it compiled and ran correctly.

Branch	Purpose
main	Stable, working version of the project (default branch)
patient_branch	Patient module: registration, triage queue, search and update
reports_branch	Performance and analytics report
console_graphics_effects	Console colours, animations and charts
comment_branch	Function and file comments
testing_and_debugging	Testing and bug fixing

Active branches in the GitHub repository

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
testing_and_debugging 	 1 hour ago		0	3	 ...
comment_branch 	 10 hours ago		0	0	 ...
patient_branch 	 yesterday		2	0	 ...
console_graphics_effects 	 2 days ago		6	0	 ...
reports_branch 	 3 days ago		17	0	 ...

7.2 Best Practices Followed

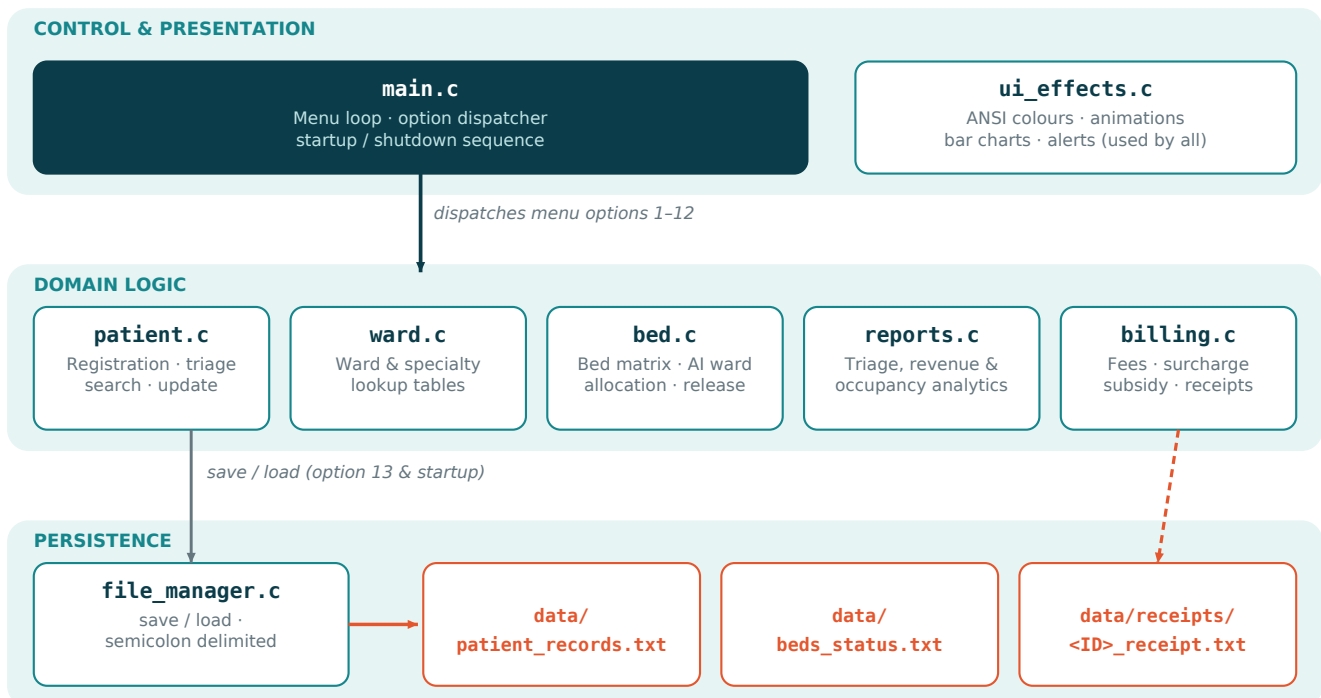
- **Repository hygiene** a `.gitignore` excludes compiled binaries (`*.exe`, `*.o`), `Code::Blocks *.depend / *.layout` files, `bin/`, `obj/` and temporary or log files, so only source and data are tracked. The `.depend` and `.layout` files had been pushed by mistake earlier in development, and were removed from the repository in the final stage of the project.
- **Feature branches** new work was done on separate branches and merged into `main` only when it was working, which kept `main` stable.
- **Small, frequent commits** changes were committed one feature at a time, so each commit has a single clear purpose.
- **Clear commit messages** messages describe what changed, for example “Add bed allocation with ward recommendation”, not just “update”.
- **Push only working code** code was merged into `main` after it compiled and ran correctly, so `main` always holds a working version.
- **History and backup** GitHub keeps every version, so earlier work can be restored and the project is safely backed up online.
- **Documentation in the repository** the README and this report are kept alongside the source code.

8 Implementation

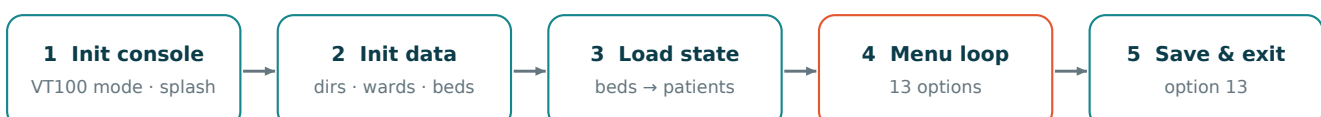
This section describes how the system is built: its architecture, the functions behind each menu option, and the data structures they operate on.

8.1 High Level Architecture

The code is split by responsibility into three layers. **main.c** only orchestrates; domain modules own the business rules and never touch files directly; **file_manager.c** is the single gateway to disk, and **ui_effects.c** isolates every terminal styling concern.



8.2 Program Lifecycle & Menu Map



#	Menu option	Handler	#	Menu option	Handler
1	Register new patient	registerPatient	8	Ward bed status tables	displayBedOccupancyMetrics
2	Patients directory	displayPatients	9	Allocate bed (AI recommendation)	allocateBed
3	Priority triage queue	displayPriorityTriageQueue	10	Discharge & release bed	releaseBed
4	Search patient	searchPatient	11	Performance reports	generatePerformanceReport
5	Update patient	updatePatientRecord	12	Bill & export receipt	processBillCalculation
6	Doctor specialties	displaySpecialties	13	Save data & exit	savePatientRecords + saveBedStatus
7	Hospital wards	displayWards			

8.3 Core Business Rules

Rule	Logic implemented
Triage ordering	Level 3 (Critical) → 2 (Urgent) → 1 (Normal). Stable bubble sort on a <i>temporary copy</i> , so arrival order is kept for ties and the directory is untouched.
Ward recommendation	Critical → ICU (4); age < 12 → Paediatric (2); Urgent → Surgical (3); otherwise General (1). Advisory only: the user may pick another ward.
Bed allocation	First fit: the lowest free bed ID in the chosen ward is assigned. Ward counters and bed record are updated together.
Billing	Gross = base fee + surcharge + (ward daily rate × days). Surcharge = 0 % / 20 % / 50 % of base fee for Level 1 / 2 / 3. 15 % subsidy on gross if age < 5 or > 65. Final = Gross – Subsidy.
Wait time estimate	Critical = 0 min. Otherwise (noncritical patients of the same specialty) × average consultation minutes; falls back to all noncritical patients if that queue is empty.

8.4 Implemented Functions (58)

Grouped by source file. † = defined and available, but not called by the current menu flow.

main.c		2 functions
main()	Startup sequence, menu loop, dispatch, save on option 13	
displayMenu()	Draws the boxed 13 option main menu	
patient.c		13 functions
clearInputBuffer()	Discards leftover characters in stdin	
readIntBounded()	Reads an integer within min-max, reprompts on error	
readStringNonEmpty()	Reads a line and rejects empty input	
readGender()	Menu based gender selection (Male / Female)	
isDuplicatePatient()	Detects an existing name + contact pair	
registerPatient()	Full intake, triage level, optional ward and bed auto allocation	
displayPatients()	Directory table in registration order	
findPatientByID()	Returns pointer to a patient record, or NULL	
getPatientCount() †	Returns the number of registered patients	
sortPatientsByPriority()	Stable bubble sort by emergency level (3 → 1)	
displayPriorityTriageQueue()	Colour coded priority queue with sorting animation	
searchPatient()	Search by ID or case insensitive partial name	
updatePatientRecord()	Edit name, age, gender, contact, condition, urgency	

ward.c		4 functions
initWardsAndSpecialties()	Seeds 4 wards and 7 specialties with rates, beds, fees	
displayWards()	Ward table with live available bed count	
displaySpecialties()	Specialty directory with fee, time and daily cap	
getWardByID()	Returns pointer to a Ward, or NULL	
bed.c		10 functions
initBeds()	Builds 45 beds: 101-120, 201-210, 301-310, 401-405	
displayBeds()	Combined bed view: matrix, cards and ward tables	
displayBedMatrix()	2D grid, green = free, red = occupied	
displayGraphicalBedCards()	5 wide colour coded bed cards per ward	
displayBoxedWardBedStatus() †	Boxed per ward status panels	
findAvailableBed()	First free bed in a ward, or -1 if full	
recommendWard()	Rule based ward suggestion from urgency and age	
displayBedOccupancyMetrics()	Occupancy summary per ward (menu option 8)	
allocateBed()	Recommend, choose ward, assign first free bed	
releaseBed()	Discharge patient, free bed, restore counters	
billing.c		9 functions
calculateWardCost()	Daily rate × days (0 when days ≤ 0)	
calculateEmergencySurcharge()	0 % / 20 % / 50 % of base fee by urgency	
calculateGrossTotal() †	Simple ward cost + base charge (earlier version)	
calculateGrossTotalV4()	Base fee + surcharge + ward cost	
calculateAgeSubsidy()	15 % discount for age < 5 or > 65	
calculateFinalAmount() †	Returns gross unchanged (earlier version)	
calculateFinalAmountV4()	Gross total – subsidy discount	
calculateEstimatedWaitTime()	Queue length × specialty consultation time	
processBillCalculation()	Interactive bill, on screen statement, receipt file export	
reports.c		1 function
generatePerformanceReport()	Triage split, revenue and discounts, ward occupancy, top bill	
file_manager.c		5 functions
initDirectoriesAndFiles()	Ensures the data/ folder exists (cross platform mkdir)	
savePatientRecords()	Writes count + one semicolon delimited line per patient	
loadPatientRecords()	Parses records with strtok back into patients[]	
saveBedStatus()	Writes all 45 bed records	
loadBedStatus()	Restores beds and decrements ward available counters	

ui_effects.c		14 functions
milliSleep() (static)	Cross platform millisecond delay	
enableVTMode()	Turns on ANSI colours and UTF-8 on Windows	
displayStartupScreen()	Animated splash with progress bar	
drawBoxHeader() †	Double line boxed section header	
pauseConsole()	Waits for Enter before returning to menu	
showProgressBar()	Progress bar for file operations	
showLoadingSpinner()	Timed spinner for billing, sorting, reports	
showBedScanningAnimation()	Ward bed scan animation	
showBedAllocationVisual()	Bed assigned confirmation banner	
showTriageAlert()	Urgency alert box (red / yellow / green)	
showTriageQueueSortingAnimation()	Queue sorting animation	
showHospitalStatusDisplay()	Live patients / beds occupancy box	
drawBarChart()	Colour coded horizontal block bar chart	
displayColorLegend()	Colour key legend	

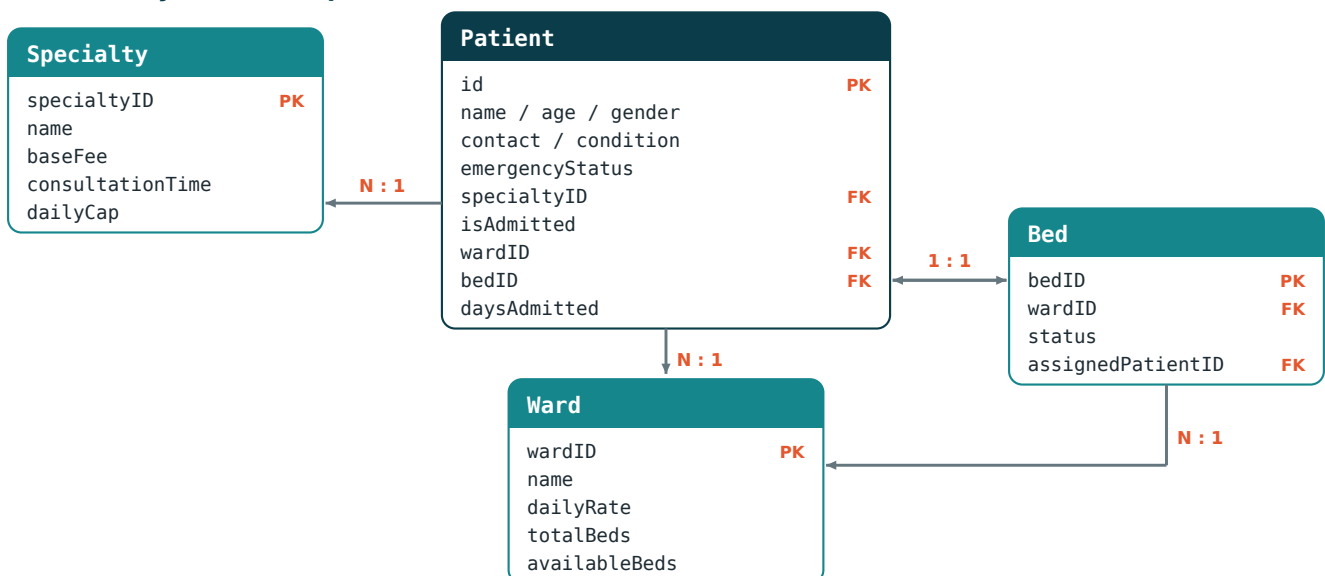
8.5 Data Structure Designs

Four typedef struct records model the domain. They are stored in **statically allocated global arrays** (no malloc), which keeps memory use predictable and avoids leaks. Cross references use **ID values instead of pointers** so every relationship can be written to and restored from text files.

8.5.1 Global storage

Variable	Type	Capacity	Notes
patients[]	Patient	100 (MAX_PATIENTS)	Filled sequentially; patientCount is the live length (≈ 17.6 KB)
beds[]	Bed	45 (TOTAL_BEDS_IN_HOSPITAL)	One entry per physical bed, fixed at startup (≈ 1.2 KB)
wards[]	Ward	4 (MAX_WARDS)	Lookup table; availableBeds is a live counter
specialties[]	Specialty	7 (MAX_SPECIALTIES)	Read only lookup for fees and consultation times

8.5.2 Entity relationships



8.5.3 Patient (patient.h)

Field	Type	Description
id	char[20]	Unique key generated as PAT-%04d, starting at PAT-1001
name	char[50]	Full name
age	int	Years, validated 1–120; drives ward recommendation and subsidy
gender	char[10]	“Male” or “Female”
contact	char[20]	Telephone number (with name, forms the duplicate key)
condition	char[50]	Medical condition / visit reason
emergencyStatus	int	Triage level: 1 = Normal OPD, 2 = Urgent, 3 = Critical
specialtyID	int	Foreign key to specialties[], 1–7
isAdmitted	int	1 = ward admission, 0 = outpatient
wardID	int	Foreign key to wards[], 1–4; 0 for OPD
bedID	int	Foreign key to beds[] (e.g. 401); 0 when no bed
daysAdmitted	int	Length of stay in days (0–365); feeds the ward cost

8.5.4 Bed, Ward and Specialty

Struct	Field	Type	Description
Bed	bedID	int	Unique ID by ward block: 101–120, 201–210, 301–310, 401–405
	wardID	int	Owning ward, 1–4
	status	int	0 = Available, 1 = Occupied
	assignedPatientID	char[15]	Patient ID, or the string “None” when free
Ward	wardID	int	Primary key, 1–4
	name	char[30]	General, Paediatric, Surgical, ICU
	dailyRate	float	Accommodation cost in LKR per day (3,000 General → 25,000 ICU)
	totalBeds / availableBeds	int	Fixed capacity (20 / 10 / 10 / 5) and live free bed counter
Specialty	specialtyID	int	Primary key, 1–7
	name	char[40]	e.g. Cardiology, Neurology, ENT
	baseFee	float	Consultation fee in LKR (1,500 – 5,000)
	consultationTime / dailyCap	int	Average minutes (15–30) and daily patient limit (10–30)

8.6 Persistence File Formats

```

data/patient_records.txt
<count>
id;name;age;gender;contact;condition;emergencyStatus;specialtyID;isAdmitted;wardID;bedID;daysAdmitted

data/beds_status.txt (45 lines)
bedID;wardID;status;assignedPatientID

data/receipts/<patientID>_receipt.txt
Human readable itemised bill, one file per patient, overwritten on each calculation

```

Design rationale. Linear search is used for lookups because the data set is small (≤ 100 patients, 45 beds). Semicolons separate fields so names and conditions containing spaces load correctly with `strtok`.

9 Conclusion

The Smart Hospital & Resource Allocation System shows how core C concepts (structs, arrays, functions, file handling and input validation) combine into a working, menu driven application. Its eight modules cover the whole patient journey: registration and triage, ward recommendation and bed allocation, billing with emergency surcharges and age subsidies, and analytics on triage, revenue and occupancy.

Data survives between runs through semicolon delimited text files, and separating logic, storage and presentation keeps each module small enough to test and maintain. The design has known limits, listed in section 3.1: fixed capacity, single user operation and saving only on exit. These are natural starting points for future work such as dynamic memory or a database back end.

10 Appendix

10.1 GitHub Repository

Repository URL	https://github.com/kwh440/Smart-Hospital-Resource-Allocation-System
Repository name	Smart-Hospital-Resource-Allocation-System
Branch	main (default), plus feature branches for modules and tasks (see 7.1)
Project	Smart Hospital & Resource Allocation System (C)

10.2 Source Code

Files	Role
main.c	Entry point, menu loop and dispatcher
patient.c / .h	Patient intake, validation helpers, triage queue, search and update
ward.c / .h	Ward and specialty lookup tables
bed.c / .h	Bed matrix, ward recommendation, allocation and release
billing.c / .h	Billing formulas, wait time estimate and receipt export
reports.c / .h	Performance and analytics report
file_manager.c / .h	Save and load of patient and bed data
ui_effects.c / .h	Console colours, animations and charts

10.3 Output Screenshots

Screenshots taken from a sample run of the program.

Figure 1 · Startup animation and main menu

```
SMART HOSPITAL SYSTEM
  Initializing System...
[=====] 100%
  SYSTEM READY [OK]

[File Storage] Loading bed matrix status... [=====] 100%
[File Storage] Successfully loaded bed matrix from the database.

[File Storage] Loading patient records... [=====] 100%
[File Storage] Successfully loaded 5 patient record(s) from the database.

SMART HOSPITAL RESOURCE SYSTEM

[ PATIENT MANAGEMENT & TRIAGE ]
1. Register New Patient
2. Display Patients Directory
3. Display Priority Triage Queue
4. Search Patient Record
5. Update Patient Record

[ WARD & BED MANAGEMENT ]
6. Display Doctor Specialties (7 Specialties)
7. Display Hospital Wards (4 Wards)
8. Display Ward Bed Status Tables
9. Allocate Bed (With AI Recommendation)
10. Discharge Patient & Release Bed

[ FINANCIALS & ANALYTICS ]
11. Generate Performance Reports & Analytics
12. Calculate Admission Bill & Export Receipt
13. Save Data & Exit System

Enter your choice (1-13):
```

WHAT IT SHOWS

- Animated splash screen with a progress bar.
- Saved bed matrix and patient records are loaded from disk (5 patient records restored).
- The 13 option main menu, grouped into Patient Management & Triage, Ward & Bed Management and Financials & Analytics.

Figure 2 · Patient registration (option 1)

```
11. Generate Performance Reports & Analytics
12. Calculate Admission Bill & Export Receipt
13. Save Data & Exit System

Enter your choice (1-13): 1

--- PATIENT REGISTRATION FORM (PAT-1006) ---
Enter Patient Name: John Do
Enter Contact Number: 0972551980
Enter Age (1 to 120 years): 30
Select Gender (1 = Male, 2 = Female): 1

=====
DOCTOR SPECIALTIES DIRECTORY
=====
Specialty ID  Specialty Name      Base Fee (LKR)  Avg Time (Mins)  Daily Cap
-----
1  General Practice (GPO)         1500.00         15             30
2  Paediatrics                   2500.00         20             20
3  Cardiology                    4500.00         30             12
4  Neurology                    5000.00         20             10
5  Orthopaedics                 3500.00         25             25
6  Dermatology                  3000.00         15             30
7  ENT (Ear, Nose, Throat)       3200.00         15             30
=====
Select Doctor Specialty ID (1 to 7): 6
Enter Medical Condition / Visit Reason: Show reports
Enter Emergency Status (1 = Normal GPO, 2 = Urgent, 3 = Critical Emergency): 1
Is Patient Admitted to Ward? (1 = Yes, 0 = No / GPO): 1

=====
HOSPITAL WARDS LIST
=====
Ward ID  Ward Name      Rate (LKR/Day)  Total Beds  Available Beds
-----
1  General Ward   3000.00         20          19
2  Paediatric Ward 6000.00         10           9
3  Surgical Ward 12000.00        10          10
4  ICU           25000.00         5           3
=====
Select Ward ID (1 to 4): 3

Allocate a bed in selected Ward now? (1 = Auto-Allocate Bed, 0 = Skip Bed Allocation): 1

[Bed Matrix] Scanning Ward #3 Bed Availability: [ Bed-01.. Bed-02.. Bed-03.. Bed-04.. Bed-05.. ] -> AVAILABLE BED FOUND [OK]

=====
BED ALLOCATION CONFIRMATION
=====
BED ID ASSIGNED: 301
PATIENT ID: PAT-1006
PATIENT NAME: John Do
STATUS: RESERVES & OCCUPIED [CONFIRMED]

Enter Initial Days Admitted (e.g. 1-30): 2

Processing Patient Registration... [OK]

=====
[1] NORMAL TRIAGE NOTICE (LEVEL 1)
=====
Patient: John Do
Assigned: STANDARD GPO QUEUE

[Success] Patient 'John Do' registered successfully with assigned ID: PAT-1006
[Press ENTER to return to menu...]
```

WHAT IT SHOWS

- Intake form: name, contact number, age and gender.
- Specialty and ward directories are shown so the user can choose.
- Auto allocation finds a free bed (301) and shows a confirmation box.
- Triage notice and a success message with the new ID, PAT-1006.

Figure 3 · Patients directory (option 2)

SMART HOSPITAL RESOURCE SYSTEM							
[PATIENT MANAGEMENT & TRIAGE]							
1. Register New Patient							
2. Display Patients Directory							
3. Display Priority Triage Queue							
4. Search Patient Record							
5. Update Patient Record							
[WARD & BED MANAGEMENT]							
6. Display Doctor Specialties (7 Specialties)							
7. Display Hospital Wards (4 Wards)							
8. Display Ward Bed Status Tables							
9. Allocate Bed (With AI Recommendation)							
10. Discharge Patient & Release Bed							
[FINANCIALS & ANALYTICS]							
11. Generate Performance Reports & Analytics							
12. Calculate Admission Bill & Export Receipt							
13. Save Data & Exit System							

Enter your choice (1-13): 2

REGISTERED PATIENTS DIRECTORY							
Patient ID	Name	Age	Gender	Contact	Status	Ward	Bed
PAT-1001	Kamal Perera	50	Male	0111111110	Normal	4	401
PAT-1002	Amara U.	12	Male	0112344411	Urgent	0	0
PAT-1003	Daham Anuradha	25	Male	NaN	Normal	1	101
PAT-1004	Nanda Umali	40	Female	NaN	Critical	4	402
PAT-1005	A. Kamala V.	79	Female	0112114434	Critical	2	201

[Press ENTER to return to menu...]

WHAT IT SHOWS

- All registered patients in registration order.
- Columns: patient ID, name, age, gender, contact, status, ward and bed.
- The status is colour coded (Normal, Urgent, Critical).
- Outpatients show ward 0 and bed 0.

Figure 4 · Priority triage queue (option 3)

SMART HOSPITAL RESOURCE SYSTEM							
[PATIENT MANAGEMENT & TRIAGE]							
1. Register New Patient							
2. Display Patients Directory							
3. Display Priority Triage Queue							
4. Search Patient Record							
5. Update Patient Record							
[WARD & BED MANAGEMENT]							
6. Display Doctor Specialties (7 Specialties)							
7. Display Hospital Wards (4 Wards)							
8. Display Ward Bed Status Tables							
9. Allocate Bed (With AI Recommendation)							
10. Discharge Patient & Release Bed							
[FINANCIALS & ANALYTICS]							
11. Generate Performance Reports & Analytics							
12. Calculate Admission Bill & Export Receipt							
13. Save Data & Exit System							

Enter your choice (1-13): 3

Sorting Triage Priority Queue (Critical -> Urgent -> Normal) [Ok]

[LEGEND] [AVAILABLE / SUCCESS] [URGENT / WARNING] [OCCUPIED / CRITICAL] [HEADINGS]

EMERGENCY TRIAGE PRIORITY QUEUE							
#	Patient ID	Name	Priority	Age	Medical Condition	Ward	Bed
1	PAT-1004	Nanda Umali	CRITICAL	40	Vehicle Accident	4	402
2	PAT-1005	A. Kamala V.	CRITICAL	79	Show Dermatology Report	2	201
3	PAT-1002	Amara U.	URGENT	12	Eye problem	0	0
4	PAT-1001	Kamal Perera	NORMAL	50	Dengue/fewer	4	401
5	PAT-1003	Daham Anuradha	NORMAL	25	Vehicle Accident	1	101

[Press ENTER to return to menu...]

WHAT IT SHOWS

- A sorting animation orders the queue Critical, Urgent, Normal.
- A colour legend explains the status colours.
- The table lists rank, patient, priority, age, medical condition, ward and bed.
- Critical patients come first; patients of equal priority keep their registration order.

Figure 5 · Performance reports and analytics (option 11)

```

13. Save Data & Exit System
Enter your choice (1-13): 11
Generating Hospital Performance Analytics Report... [OK]

HOSPITAL REAL-TIME STATUS
Total Active Patients: 6
Bed Occupancy Count: 5 / 45
Bed Utilization Rate: 11.1 %

SMART HOSPITAL SYSTEM - PERFORMANCE & ANALYTICS REPORT

1. PATIENT REGISTRATION & TRIAGE SUMMARY
Total Patients Registered : 6
- Level 1 (Normal QPO) : 3 (50.0%)
- Level 2 (Urgent) : 1 (16.7%)
- Level 3 (Critical) : 2 (33.3%)

[Triage Distribution Chart]
Normal QPO : 50.0% (3)
Urgent Priority : 16.7% (1)
Critical Emergency : 33.3% (2)

2. FINANCIAL REVENUE & DISCOUNT SUMMARY
Total Base Consultation Fees : LKR 16700.00
Total Emergency Surcharges : LKR 2890.00
Total Ward Stay Costs : LKR 258000.00
TOTAL GROSS REVENUE : LKR 277590.00
TOTAL DISCOUNTS GRANTED : LKR -1575.00 (-15% Age Subsidy)
TOTAL NET REVENUE EARNED : LKR 276015.00

3. BED OCCUPANCY & UTILIZATION METRICS SUMMARY
Ward ID | Ward Name | Total Beds | Occupied | Available | Occupancy %
1 | General Ward | 20 | 1 | 19 | 5.0 %
2 | Paediatric Ward | 10 | 1 | 9 | 10.0 %
3 | Surgical Ward | 10 | 1 | 9 | 10.0 %
4 | ICU | 5 | 2 | 3 | 40.0 %
TOTAL | All Wards Combined | 45 | 5 | 40 | 11.1 %

[Ward Occupancy Utilization Chart]
General Ward : 5.0% (1)
Paediatric Ward : 10.0% (1)
Surgical Ward : 10.0% (1)
ICU : 40.0% (2)

4. HIGHEST-PAYING PATIENT RECORD
Patient ID : PAT-1001
Patient Name : Kamal Perera
Age : 50 Years
Urgency Level : Normal
Ward Admission : ICU
Highest Total Bill : LKR 154500.00

[Press ENTER to return to menu...]

```

WHAT IT SHOWS

- Real time status: 6 active patients, 5 of 45 beds occupied (11.1 %).
- Triage summary with a colour coded distribution chart.
- Revenue summary: gross revenue, age subsidy discounts and net revenue in LKR.
- Bed occupancy table and utilisation chart for each ward.
- Highest paying patient record.

Figure 6 · Bill calculation and receipt export (option 12)

```

10. Discharge Patient & Release Bed

[ FINANCIALS & ANALYTICS ]
11. Generate Performance Reports & Analytics
12. Calculate Admission Bill & Export Receipt
13. Save Data & Exit System

Enter your choice (1-13): 12

Enter Patient ID for bill calculation (e.g., PAT-1001): PAT-1001

[Verification Prompt] Is Days Admitted correct (6 days)? (1 = Yes, 0 = Update): 0
Enter updated/correct Days Admitted: 4
[Update] Days Admitted updated to 4 days.

Calculating billing breakdown & subsidies... [OK]

SMART HOSPITAL ADMISSION & BILL

Patient ID: PAT-1001
Patient Name: Kamal Perera
Age: 50 Years (Standard Rate)
Specialty: Cardiology
Assigned Ward: ICU (Bed #401)
Urgency Level: Level 1 (Normal)

Base Consultation Fee: LKR 4500.00
Emergency Surcharge: LKR 0.00 (0%)
Ward Stay Cost (4 Days): LKR 100000.00

Gross Total Bill: LKR 104500.00
Age Subsidy Discount: LKR - 0.00 (0%)

Final Payable Amount: LKR 104500.00
Estimated Waiting Time: 30.00 mins

[Receipt Export] Itemized bill saved to the database.
[Press ENTER to return to menu...]

```

WHAT IT SHOWS

- Days Admitted is verified and can be corrected (6 updated to 4 days).
- Itemised bill: base consultation fee, emergency surcharge and ward stay cost.
- Age subsidy discount and the final payable amount.
- The receipt is exported to a text file.

10.4 Project Details

Project title	Smart Hospital & Resource Allocation System
Language	C (C99 / C11)
IDE	Code::Blocks (v20.03 or later recommended)
Version control	GitHub
Operating system	Windows 10 / 11 (also Linux and macOS)
Repository branch	main (default), with feature branches

11 Challenges Faced

As a first year student building a multifile C program, I met the following challenges:

Challenge	How I overcame it
Handling keyboard input safely	Leftover characters in the input buffer skipped prompts and bad input caused loops. I wrote <code>clearInputBuffer</code> , <code>readIntBounded</code> and <code>readStringNonEmpty</code> and used them everywhere.
Keeping patient and bed data consistent	A patient stores a ward and bed ID while the bed stores the patient ID. I updated both sides in one function when allocating or releasing a bed, and also updated the ward's free bed counter.
Saving and loading text files	Names and conditions contain spaces, so I used semicolons as separators and <code>strtok</code> to read them back, and loaded the beds before the patients so ward counters stay correct.
Sorting the triage queue	Sorting the main array would have destroyed the registration order, so I sorted a temporary copy with a stable bubble sort.
Showing colours on the Windows console	ANSI colours did not appear by default, so I enabled virtual terminal mode in <code>enableVTMode</code> and used conditional compilation for platform differences.
Learning Git and managing time	Working module by module with regular commits made progress easy to track and deadlines easier to manage.

What I learned. The project strengthened my understanding of structs, arrays, pointers, functions, file handling and modular design in C, and showed me the value of planning, testing in small steps and using version control.

12 System Requirements

The application is a pure command line program that needs only a C compiler and a terminal.

Language and standard	C, C99 / C11 compliant (ANSI C standard)
Compiler	GCC 4.8+, MinGW-w64, Clang 3.8+ or MSVC 2017+
IDE / tools	Code::Blocks v20.03 or later, recommended with GCC / MinGW; or a command line terminal (PowerShell, Bash, CMD)
Operating system	Windows 10 / 11, Linux or macOS
Target binary	Pure CLI console application
Terminal	ANSI VT100 Virtual Terminal colour support (UTF-8 for box drawing characters)
Storage	Read / write access to the local <code>./data/</code> directory (<code>patient_records.txt</code> , <code>beds_status.txt</code>); billing receipts are written to <code>data/receipts/</code>
Hardware	No special hardware; about 20 KB of static data at run time

Standard C library dependencies (built in, no external libraries)

Header	Used for	Functions used
stdio.h	Console and file I/O	printf, scanf, fgets, file I/O
stdlib.h	Process control and conversion	system, atoi
string.h	String manipulation	strcmp, strcpy, strlen, strtok, strcspn
ctype.h	Character classification	tolower

Build and run

For Windows (PowerShell / Command Prompt):

```
# 1. Build (Compile all C files from src/ into bin/hospital_app.exe)
gcc -Wall -std=c99 -I src src/*.c -o bin/hospital_app.exe
# 2. Run
.\bin\hospital_app.exe
```

For Linux / macOS (Terminal):

```
# 1. Build
gcc -Wall -std=c99 -I src src/*.c -o hospital
# 2. Run
./hospital
```